SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Deepak	**Reg. No.:**	192411021
Section / Batch:	C-2024	**Date:**	22-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I Deepak V-192411021 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Deepak

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

Department of Computer Science Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Case Study Report

Q8. Activity Selection – Scheduling Optimization

Activity Selection – Scheduling Optimization Using a Greedy Algorithm

Submitted by

Deepak

Register No: 192411021

B.E. Computer Science Engineering – III Year

Course Code: DAA

Academic Year 2026

Table of Contents

1. Introduction

Algorithm design is the process of defining a clear, step-by-step procedure to solve a computational problem in an efficient and correct manner. In many real-world situations, a problem does not simply require any working solution — it requires the best possible solution under a given set of constraints. Problems of this nature are known as optimization problems, where the objective is to either maximize or minimize a specific quantity, such as cost, time, distance, or in this case, the number of tasks that can be completed.

Scheduling is one of the most common and practically important optimization problems encountered in computer science and everyday business operations. Whenever a limited resource, such as a meeting room, a machine, or a processor, must be shared among multiple competing requests, an efficient scheduling strategy is required to make the best use of the available time.

This report focuses on a specific and classic optimization problem known as the Activity Selection Problem. The objective of this problem is to select the maximum possible number of activities that do not overlap with one another, given a set of activities each defined by a start time and a finish time. Throughout this report, the problem is explained using the running example of a software company that must schedule client meetings in a single conference room.

2. Real-World Problem

Consider a software company that has only one conference room available for meeting clients. Several clients approach the company and request meetings at different times of the day. Since the same conference room cannot be used by two clients at the same time, any two meetings whose time intervals overlap cannot both be accepted.

The company's objective is straightforward: accept as many client meetings as possible during the available working hours, without allowing any two accepted meetings to clash. This directly translates into the Activity Selection Problem, where each client meeting is treated as an “activity” with a defined start time and finish time.

Why Simple Approaches Fail

It may seem intuitive to accept meetings in the order in which clients request them, or to always pick the meeting that starts earliest, or the meeting with the shortest duration. However, none of these strategies reliably produce the maximum number of non-overlapping meetings.

For example, selecting the earliest-starting meeting first may lock the room for a long duration, blocking out several shorter meetings that could otherwise have been accommodated. Similarly, choosing the shortest meeting does not guarantee that the remaining time will be used optimally, since a short meeting could still occupy a time slot that overlaps with many other requests. As will be shown later in this report, the strategy that reliably works is selecting the meeting that finishes earliest among the remaining compatible options.

3. Problem Statement

Given a set of activities, where each activity i has a start time s(i) and a finish time f(i), the objective is to select the maximum number of activities such that no two selected activities overlap in time.

> *Compatibility Condition: An activity is compatible with the previously selected activity if the start time of the current activity is greater than or equal to the finish time of the previously selected activity, i.e., start(current) ≥ finish(previous).*

In the context of the running example, each activity represents a client meeting, and "selecting" an activity means the company agrees to host that meeting in the conference room.

4. Example Dataset

The following table lists the meeting requests received by the company, in the order the clients originally requested them. Each meeting is identified by a letter and defined by its start time and finish time (in hours, on a 24-hour scheduling window).

Meeting	Start Time	Finish Time
A	1	4
B	3	5
C	0	6
D	5	7
E	3	9
F	5	9
G	6	10
H	8	11
I	8	12
J	2	14
K	12	16

5. Algorithm Used

The Activity Selection Problem is solved using a Greedy Algorithm. A greedy algorithm builds up a solution step by step, and at each step it makes the choice that looks best at that moment, without reconsidering earlier choices. For this problem, the greedy strategy is:

Greedy Strategy: Always select the activity that finishes earliest among the currently available (compatible) activities.

Algorithm Steps

- Step 1: Sort all activities according to their finishing times, in increasing order.
- Step 2: Select the first activity in the sorted list, because it has the earliest finishing time.
- Step 3: Set its finish time as the current finish time.
- Step 4: Examine the remaining activities in sorted order, one at a time.
- Step 5: Select an activity if its start time is greater than or equal to the finish time of the previously selected activity.
- Step 6: Update the current finish time to the finish time of the newly selected activity.
- Step 7: Continue this process until all activities have been considered.

6. Why Finishing Time?

The greedy strategy is based on finishing time rather than starting time or duration because the finishing time of an activity determines how much time remains available for scheduling subsequent activities. Selecting the activity that finishes earliest leaves the largest possible remaining time window in which future activities can be scheduled.

Starting time alone does not indicate how long the resource will remain occupied, and duration alone does not indicate when the resource becomes free again. Finishing time captures exactly the information needed: the earliest possible moment at which the room becomes available for the next meeting.

Analogy: If one meeting ends at 4 PM and another ends at 6 PM, choosing the meeting that ends at 4 PM leaves more time in the day for additional meetings to be scheduled afterward.

7. Manual Execution / Step-by-Step Process

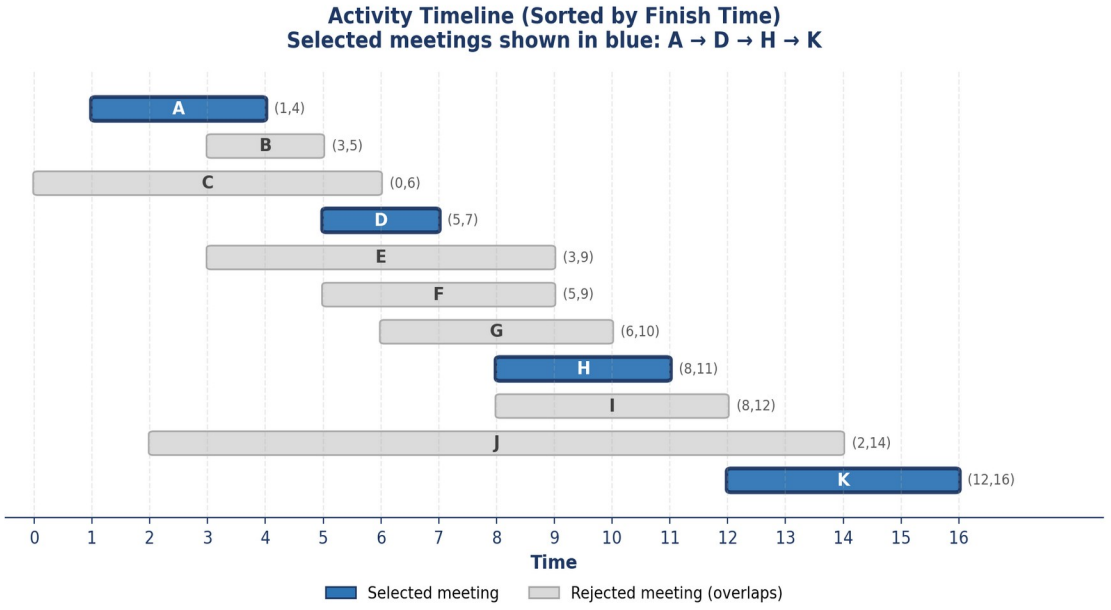
The first step of the algorithm is to sort the eleven meetings by their finishing time in increasing order. After sorting, the meetings appear in the following order:

```
A (1,4)   B (3,5)   C (0,6)   D (5,7)   E (3,9)
F (5,9)   G (6,10)  H (8,11) I (8,12)  J (2,14)
K (12,16)
```

The greedy selection process begins with meeting A, since it finishes earliest at time 4. Every subsequent meeting is then checked against the finish time of the most recently selected meeting.

Step	Meeting	Start	Finish	Decision	Reason
1	A	1	4	Selected	First activity in sorted order (earliest finish)
2	B	3	5	Rejected	Start (3) < current finish (4)
3	C	0	6	Rejected	Start (0) < current finish (4)
4	D	5	7	Selected	Start (5) ≥ current finish (4)
5	E	3	9	Rejected	Start (3) < current finish (7)
6	F	5	9	Rejected	Start (5) < current finish (7)
7	G	6	10	Rejected	Start (6) < current finish (7)
8	H	8	11	Selected	Start (8) ≥ current finish (7)
9	I	8	12	Rejected	Start (8) < current finish (11)
10	J	2	14	Rejected	Start (2) < current finish (11)
11	K	12	16	Selected	Start (12) ≥ current finish (11)

The complete process can also be visualized on a timeline, as shown below.



Final Selected Meetings: A → D → H → K | Total = 4 meetings

8. Why Is the Greedy Method Optimal?

The greedy method works correctly for the Activity Selection Problem because this problem satisfies what is known as the greedy-choice property, meaning that a locally optimal choice at each step leads to a globally optimal solution. This can be understood using a simple line of reasoning called the exchange argument.

- Consider any optimal solution to the problem.
- Its first selected activity may not necessarily be the activity that finishes earliest overall.
- Replace this first activity with the activity that finishes earliest.
- Because the earliest-finishing activity ends no later than the original first activity, making this replacement does not reduce the amount of time remaining for scheduling the rest of the activities.
- Therefore, an optimal solution can always be rearranged to begin with the earliest-finishing activity, without making the solution any worse.
- The same reasoning can then be applied again to the remaining activities after the first one is fixed.
- Repeating this reasoning at every step shows that the greedy strategy produces a solution that is at least as good as any optimal solution — hence the greedy strategy is itself optimal.

It is important to note that this optimality guarantee applies specifically to the unweighted Activity Selection Problem, where the only goal is to maximize the count of selected activities. It does not automatically extend to every scheduling problem, especially those involving priorities or profits, as discussed later in the Limitations section.

9. Pseudocode

```
ACTIVITY-SELECTION(activities)
    sort activities by increasing finish time
    select first activity
    lastFinish = finish time of first activity

    for each remaining activity:
        if start time >= lastFinish:
            select activity
            lastFinish = finish time of activity

    return selected activities
```

10. Python Implementation

The greedy algorithm described above is implemented in Python as follows:

```
def activity_selection(activities):
    activities.sort(key=lambda x: x[2])

    selected = []
    last_finish = 0

    for activity in activities:
        name, start, finish = activity

        if start >= last_finish:
            selected.append(activity)
            last_finish = finish

    return selected


activities = [
    ("A", 1, 4),
    ("B", 3, 5),
    ("C", 0, 6),
    ("D", 5, 7),
    ("E", 3, 9),
    ("F", 5, 9),
    ("G", 6, 10),
    ("H", 8, 11),
    ("I", 8, 12),
    ("J", 2, 14),
    ("K", 12, 16)
]

selected = activity_selection(activities)

print("Selected Meetings:")
for activity in selected:
    print(activity[0], ":", activity[1], "-", activity[2])

print("Maximum number of meetings:", len(selected))
```

11. Expected Output

```
Selected Meetings:
A : 1 - 4
D : 5 - 7
H : 8 - 11
K : 12 - 16

Maximum number of meetings: 4
```

[INSERT PROGRAM SCREENSHOT HERE]

[INSERT OUTPUT SCREENSHOT HERE]

12. Complexity Analysis

The efficiency of the Activity Selection algorithm is determined by two main steps: sorting the activities and then scanning through them once.

Step	Time Complexity
Sorting the activities by finish time	O(n log n)
Single scan to select compatible activities	O(n)
Overall time complexity	O(n log n)
If activities are already sorted by finish time	O(n)
Space complexity (storing selected activities)	O(n)

Since sorting dominates the overall running time, the algorithm scales well even as the number of meetings grows large. Doubling the number of meetings does not double the running time; it grows only slightly faster than linearly, which makes this approach practical for real scheduling systems handling hundreds or thousands of requests.

13. Applications

- Client meeting scheduling in a shared conference room.
- Classroom scheduling in an academic institution.
- Conference room and auditorium allocation for events.
- Job scheduling on a single machine or server.
- Event planning where a single venue hosts multiple events.
- Appointment scheduling in clinics, salons, and service centers.
- General resource allocation problems with a single shared resource.

- CPU or task scheduling in simplified single-processor cases.

14. Advantages

- Simple and easy to implement.
- Produces an optimal solution for the Activity Selection Problem.
- Efficient for large datasets due to O(n log n) complexity.
- Requires only sorting followed by a single linear scan.
- Easy to understand and explain to others.
- Useful across many real-world scheduling scenarios.

15. Limitations

- The greedy strategy does not work optimally for every scheduling problem.
- It specifically guarantees optimal results for the Activity Selection Problem, where the objective is to maximize the number of non-overlapping activities.
- If activities have different priorities, profits, or weights attached to them, a different algorithm such as Weighted Interval Scheduling using Dynamic Programming may be required instead.
- The basic version of this problem assumes only one shared resource, such as a single meeting room.

16. Comparison with Other Approaches

Approach	Strategy	Time Complexity	Result
Brute Force	Try all possible combinations	O(2^n)	Can find the optimum but is highly inefficient
Greedy Activity Selection	Select earliest finishing compatible activity	O(n log n)	Optimal for the unweighted Activity Selection Problem
Dynamic Programming	Store solutions to overlapping subproblems	O(n^2) for a basic weighted interval version	Useful when activities have weights or profits

For the basic (unweighted) Activity Selection Problem, the greedy approach is preferred because it achieves the optimal result with far less computational effort than brute force, and without the additional overhead of storing and combining subproblem solutions required by dynamic programming.

17. Tools and Programming Techniques

- Python programming language.
- Any online Python compiler for quick testing.
- Visual Studio Code (VS Code) as the development environment.
- Sorting using Python's built-in sorted() function or list.sort() method.
- Lists and tuples for storing activity data.
- Lambda functions for sorting activities by finish time.

18. Result / Interpretation

Applying the greedy Activity Selection algorithm to the given dataset of eleven client meetings results in the selection of meetings A, D, H, and K, in that order, for a total of four non-overlapping meetings.

This result represents the maximum number of meetings that can be scheduled in the given example without any conflicts in the conference room. The selection process demonstrates how consistently choosing the meeting with the earliest finishing time at each step leaves the maximum possible remaining time for scheduling future meetings, ultimately leading to the largest possible count of accepted meetings.

19. Conclusion

The Activity Selection Problem is a classic example of a problem solved using a Greedy Algorithm. By always selecting the compatible activity with the earliest finishing time, the algorithm maximizes the number of non-overlapping activities that can be scheduled.

For the given client meeting scenario, the algorithm selects four meetings: A, D, H, and K. The algorithm has an overall time complexity of O(n log n) due to the sorting step, followed by an O(n) linear scan, making it efficient and well-suited for large scheduling datasets.

It is also important to note that the greedy approach is optimal specifically because the Activity Selection Problem satisfies the greedy-choice property, and this optimality does not automatically extend to every type of scheduling problem, particularly those involving weights or priorities.

20. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (3rd Edition). MIT Press.
- Levitin, A. Introduction to the Design & Analysis of Algorithms (3rd Edition). Pearson Education.
- Kleinberg, J., & Tardos, É. Algorithm Design. Pearson/Addison-Wesley.
- GeeksforGeeks. “Activity Selection Problem – Greedy Algorithm.” www.geeksforgeeks.org

- Programiz. “Activity Selection Problem.” www.programiz.com